LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

Pertemuan 7 — HTTP Networking & Konsumsi REST API

Nama Mahasiswa	Bastian Yusuf Iskandar
NIM	3337240088
Mata Kuliah	Pemrograman Mobile
Pertemuan	Pertemuan 7
Topik	HTTP Networking & Konsumsi REST API
Studi Kasus Project	TokoKita — mengambil data produk sungguhan dari REST API
Framework	Flutter (Dart) — package `http` & `GetX`
Alokasi Waktu	170 menit (3 SKS)

A. Tujuan Praktikum

1. **Mahasiswa mampu menggunakan package `http` untuk melakukan request ke REST API**: Mengirim HTTP GET request secara asynchronous ke endpoint publik Fake Store API (`https://fakestoreapi.com/products`).
2. **Mahasiswa mampu melakukan request GET dan POST beserta penanganan response-nya**: Mengelola komunikasi client-server dengan async/await, memeriksa header, status code, dan payload body response.
3. **Mahasiswa mampu melakukan parsing JSON menggunakan `jsonDecode` / `jsonEncode` serta mengadaptasi model class dengan `fromJson` / `toJson`** : Mengonversi JSON string mentah menjadi Map/List di Dart, memetakan perbedaan struktur data API ke properti class `Product` (Pertemuan 2), dan menjaga integritas type safety serta null safety.
4. **Mahasiswa mampu menampilkan data asynchronous menggunakan `async` / `await` dan `FutureBuilder`** : Membangun antarmuka yang reaktif terhadap status Future (`waiting` , `error` , `hasData`) dengan widget `FutureBuilder` , menampilkan indikator pemuatan (`CircularProgressIndicator`), dan me-render daftar produk riil menggunakan `ListView.builder` dan `ProductCard` .
5. **Mahasiswa mampu menerapkan penanganan error dasar (`try-catch` , pengecekan status code) pada proses networking**: Mengantisipasi kegagalan jaringan seperti hilangnya koneksi internet (`SocketException`), batas waktu habis (`TimeoutException`), kesalahan format (`FormatException`), dan error respons dari server (HTTP status 404/500), serta menyediakan tombol "Coba Lagi" yang interaktif bagi pengguna.

B. Alat dan Bahan

B.1 Perangkat & Lingkungan Pengembangan

- Laptop/PC dengan proyek **TokoKita** hasil pertemuan sebelumnya (Pertemuan 1 sampai Pertemuan 6).
- Editor kode: Visual Studio Code / Android Studio.
- Perangkat pengujian: Android Emulator / Windows Desktop / Google Chrome.
- SDK: Flutter 3.47.2 & Dart 3.13.2.
- Package `http` versi `^1.6.0` dan `get` versi `^4.7.3` pada `pubspec.yaml` .
- REST API Publik: Fake Store API (`https://fakestoreapi.com/products`).

B.2 Konfigurasi Dependensi (`pubspec.yaml`)

```
name: tokokita
description: "A new Flutter project."
publish_to: 'none'

environment:
  sdk: ^3.13.2

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  http: ^1.6.0       # Package utama untuk HTTP Networking (Langkah 1)
  get: ^4.7.3        # State Management & Routing dari Pertemuan 6

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
```

B.3 Struktur Berkas yang Dibuat / Diperbarui

No	Berkas	Status	Peran & Deskripsi Sinkronisasi
1	`lib/services/product_service.dart`	**Baru**	Service layer untuk HTTP networking (`fetchProducts` , `fetchProductDetail` , error handling, try-catch, dan mock client support).
2	`lib/models/product.dart`	Diperbarui	Menambahkan factory constructor `Product.fromJson()` dan method `toJson()` yang memetakan payload Fake Store API (`title` , `price` , `image` , `rating`) ke model class TokoKita tanpa merusak fungsionalitas dummy data Pertemuan 2.
3	`lib/screens/home_page.dart`	Diperbarui	Menggantikan dummy data lokal dengan `FutureBuilder<List<Product>>` , mengintegrasikan `CircularProgressIndicator` , UI error dengan tombol "Coba Lagi", `RefreshIndicator` (pull-to-refresh), dan tombol uji simulasi error.
4	`lib/screens/product_detail_page.dart`	Diperbarui	Mendukung pemuatan data via parameter objek `Product` maupun fetch detail langsung berdasarkan `productId` menggunakan `FutureBuilder<Product>` (Tugas Mandiri 1), serta mendukung rendering gambar jaringan `Image.network` .
5	`lib/widgets/product_card.dart`	Diperbarui	Menampilkan gambar produk riil via `Image.network` jika URL gambar tersedia, dengan fallback elegan ke ikon kategori jika URL kosong atau terjadi galat jaringan.
6	`test/widget_test.dart`	Diperbarui	Menambahkan 8 unit & widget test baru untuk Pertemuan 7 serta mempertahankan seluruh 9 pengujian Pertemuan 1-6 (Total 17 skenario lulus 100%).

C. Dasar Teori Singkat

Pada tahapan sebelumnya (Pertemuan 2 hingga Pertemuan 6), aplikasi TokoKita masih mengandalkan data dummy statis yang ditulis langsung pada kode program (`dummyProducts`). Dalam aplikasi toko daring berskala produksi, data katalog produk bersifat dinamis dan tersimpan di basis data terpusat pada server, sehingga pembaruan harga, stok, maupun produk baru dapat langsung dinikmati pengguna tanpa perlu memperbarui atau merilis ulang aplikasi di Play Store / App Store.

C.1 HTTP Networking & REST API pada Flutter

Komunikasi antara aplikasi mobile (client) dan server terjadi melalui protokol HTTP/HTTPS berbasis arsitektur REST (*Representational State Transfer*). Data ditransmisikan menggunakan method standar:
- `GET`: Mengambil atau membaca data dari server.
- `POST`: Mengirim data baru ke server.
- `PUT` / `PATCH`: Memperbarui data yang ada di server.
- `DELETE`: Menghapus data dari server.

Package `http` dari tim resmi Dart (`package:http/http.dart`) menyediakan antarmuka tingkat tinggi untuk menjalankan request HTTP:

```
final response = await http.get(Uri.parse('https://fakestoreapi.com/products'));
```

C.2 Operasi Asynchronous & Widget `FutureBuilder`

Request HTTP memiliki jeda waktu (*network latency*) yang bergantung pada kecepatan koneksi internet dan responsivitas server. Oleh karena itu, eksekusi operasi ini dilakukan secara asynchronous menggunakan tipe data `Future` dan kata kunci `async` / `await` agar antarmuka aplikasi tidak membeku (*non-blocking UI*).

Untuk menampilkan data dari `Future` secara otomatis, Flutter menyediakan widget `FutureBuilder<T>` yang memonitor siklus hidup `AsyncSnapshot<T>`:

```
  [Inisialisasi Future]
           |
           ▼
ConnectionState.waiting ──► Tampilkan CircularProgressIndicator (Loading)
           |
     ┌─────┴──────────────────────────┐
     ▼                                ▼
snapshot.hasError              snapshot.hasData
     |                                |
     ▼                                ▼
Tampilkan Pesan Error          Tampilkan ListView.builder
& Tombol "Coba Lagi"           berisi ProductCard
```

C.3 Serialisasi & Deserialisasi JSON

Data yang dikirim oleh REST API umumnya berformat teks JSON (*JavaScript Object Notation*).
- **Deserialisasi (`jsonDecode`)**: Mengubah teks string JSON menjadi tipe data Dart (`Map<String, dynamic>` atau `List<dynamic>`).
- **Mapping (`fromJson`)**: Mengonversi `Map<String, dynamic>` ke dalam objek model strongly-typed (`Product`).
- **Serialisasi (`toJson` & `jsonEncode`)**: Mengubah objek model kembali menjadi Map dan teks string JSON.

D. Langkah Kerja & Implementasi Kode

Langkah 1 — Setup Package `http`

1. Memastikan package `http: ^1.6.0` telah terdaftar pada berkas `pubspec.yaml` di bawah blok `dependencies`.

2. Menjalankan perintah terminal di direktori proyek:
   `bash flutter pub get`
3. Membuat berkas baru `lib/services/product_service.dart` yang bertindak sebagai layer *Service Repository* untuk mengisolasi logika networking dari UI widget.

Langkah 2 — Request GET Data Produk

1. Membuat fungsi asynchronous `fetchProducts()` di dalam `ProductService` yang mengirim request `http.get()` ke endpoint `https://fakestoreapi.com/products`.
2. Menggunakan `async/await` untuk menunggu respons, lalu mencetak `response.statusCode` dan preview `response.body` pada console untuk memverifikasi data yang diterima:
   ```dart
   final response = await activeClient
   .get(Uri.parse(endpoint))
   .timeout(const Duration(seconds: 15));

print('[ProductService] Response StatusCode: ${response.statusCode}');
print('[ProductService] Response Body Preview:\n${response.body.substring(0, 200)}...');
```

Langkah 3 — Menyesuaikan Model Product & Parsing JSON

1. Membuka `lib/models/product.dart` dan menambahkan factory constructor `Product.fromJson(Map<String, dynamic> json)` pada class `Product`.
2. Melakukan adaptasi dan normalisasi terhadap perbedaan field Fake Store API dengan model TokoKita:
3. Field `title` pada API dipetakan ke properti `name`.
4. Field `image` pada API dipetakan ke properti `imageUrl`.
5. Field `price` dari API (dollar) dikonversikan ke satuan Rupiah (kurs Rp 15.000) agar sinkron dengan fungsi `formatRupiah()` dan sistem keranjang TokoKita.
6. Field `stock` yang tidak disediakan oleh Fake Store API diberikan nilai fallback cerdas dari `rating['count']` atau default `20`.
7. Field `category` API ("men's clothing", "electronics", dll.) dinormalisasi ke kategori TokoKita ("Fashion", "Elektronik").
8. Mengimplementasikan method `toJson()` untuk serialisasi data:
   ```dart
   factory Product.fromJson(Map json, {bool convertCurrency = true}) {
   final num rawPrice = json['price'] as num? ?? 0.0;
   final double finalPrice = (convertCurrency && rawPrice < 1000 && rawPrice > 0)
   ? (rawPrice * 15000).roundToDouble()
   : rawPrice.toDouble();

   return Product(
   id: json['id'] as int? ?? 0,
   name: json['title'] as String? ?? json['name'] as String? ?? 'Produk Tanpa Nama',
   price: finalPrice,
   category: _normalizeCategory(json['category'] as String? ?? 'Umum'),
   stock: json['stock'] as int? ??
   (json['rating'] != null && json['rating']['count'] != null
   ? (json['rating']['count'] as num).toInt()
   : 20),
   imageUrl: json['image'] as String? ?? json['imageUrl'] as String?,
```

description: json['description'] as String?,
);
}

Map toJson() {
return {
'id': id,
'title': name,
'price': price,
'category': category,
'stock': stock,
'image': imageUrl,
'description': description,
};
}

`4. Di dalam `product_service.dart`, mem-parsing body respons menggunakan `jsonDecode`:` dart

if (response.statusCode == 200) {
final List jsonList = jsonDecode(response.body);
final List products = jsonList
.map((item) => Product.fromJson(item as Map))
.toList();
return products;
}
```

---

### Langkah 4 — Menampilkan Data Asli pada `HomePage` dengan `FutureBuilder`

1. Pada `lib/screens/home_page.dart`, menggantikan array produk dummy dengan variabel state `late Future<List<Product>> _productsFuture;`.
2. Menginisialisasi `_productsFuture` pada method `initState()` dengan memanggil `ProductService.fetchProducts()`.
3. Membangun tampilan daftar produk menggunakan widget `FutureBuilder<List<Product>>`:
4. **Kondisi Waiting**: Menampilkan indikator loading `CircularProgressIndicator` di tengah layar dengan teks status.
5. **Kondisi HasData**: Me-render data hasil fetch menggunakan `ListView.builder` dan widget `ProductCard` yang telah dirancang pada Pertemuan 4 tanpa mengubah struktur dasar kartu produk.
6. **Integrasi Gambar**: `ProductCard` dan `ProductDetailPage` diperkaya untuk merender URL gambar jaringan melalui `Image.network` dengan `loadingBuilder` dan `errorBuilder` fallback.
```

```
Expanded(
  child: FutureBuilder<List<Product>>(
    future: _productsFuture,
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting && _cachedProducts == null) {
        return const Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              CircularProgressIndicator(
                valueColor: AlwaysStoppedAnimation<Color>(Color(0xFF1976D2)),
              ),
              SizedBox(height: 18),
              Text(
                'Memuat produk dari Fake Store API ... ',
                style: TextStyle(fontWeight: FontWeight.w600),
              ),
            ],
          ),
        );
      }

      if (snapshot.hasError && _cachedProducts == null) {
        return _buildErrorState(snapshot.error.toString());
      }

      final allProducts = _cachedProducts ?? snapshot.data ?? [];
      return ListView.builder(
        itemCount: allProducts.length,
        itemBuilder: (context, index) ⇒ ProductCard(product: allProducts[index]),
      );
    },
  ),
)
```

Langkah 5 — Penanganan Error pada Pengambilan Data

1. Membungkus seluruh proses request HTTP pada `ProductService.fetchProducts()` dengan blok `try-catch` terstruktur yang menangani berbagai jenis kegagalan jaringan:
2. `SocketException` : Ditangkap saat perangkat tidak memiliki koneksi internet atau DNS server gagal merespons.
3. `TimeoutException` : Ditangkap jika latensi jaringan melebihi ambang batas 15 detik.
4. `FormatException` : Ditangkap jika format data bukan JSON valid.
5. `HttpException` : Dilempar jika server mengembalikan HTTP status selain 200 (misal 404 atau 500).
6. Menyediakan tombol **Simulasi Error** (ikon serangga/bug pada header `HomePage`) yang menembak endpoint keliru (`/invalid_endpoint_error`) untuk menguji skenario kegagalan secara interaktif.
7. Menampilkan pesan error yang ramah pengguna (*user-friendly error UI*) lengkap dengan ikon ilustrasi kegagalan dan tombol **"Coba Lagi"** yang secara otomatis memanggil ulang `_loadProducts(simulateError: false)` .

E. Hasil Pengamatan

Berikut adalah rekapitulasi hasil pengujian skenario networking pada aplikasi TokoKita:

No	Jenis Request / Skenario	Status Code / Hasil	Penjelasan & Bukti Verifikasi
1	**GET data produk dari Fake Store API**	**200 OK**	Endpoint `https://fakestoreapi.com/products` berhasil merespons dengan status 200. Panjang payload `response.body` sebesar 10.634 karakter berisi 20 produk katalog lengkap.
2	**Parsing JSON ke `Product.fromJson`**	**Berhasil**	Seluruh 20 item JSON berhasil dipetakan ke dalam objek `Product`. Field `title` dipetakan ke `name`, `image` ke `imageUrl`, `rating.count` menjadi fallback `stock`, serta harga dollar terkonversi ke Rupiah.
3	**Tampilan produk asli via `FutureBuilder`**	**Berhasil**	`FutureBuilder` secara dinamis menampilkan animasi `CircularProgressIndicator` saat request berlangsung, lalu me-render 20 kartu produk asli di `ListView.builder` lengkap dengan gambar produk dan kategori.
4	**Simulasi error (URL salah / tanpa koneksi)**	**404 Not Found / SocketException**	Saat simulasi error diaktifkan (menembak `/invalid_endpoint_error`), try-catch menangkap galat dan `FutureBuilder` menampilkan UI ramah pengguna: "*Gagal Memuat Produk: Server mengembalikan respons error dengan Status Code: 404*".
5	**Tombol Coba Lagi setelah error**	**200 OK (Pulih)**	Menekan tombol "Coba Lagi" memanggil ulang `_loadProducts(simulateError: false)`. Request kembali dikirim ke endpoint valid dan daftar produk pulih secara instan.

F. Tugas Mandiri

1. Function `fetchProductDetail(int id)` pada `ProductDetailPage`

Implementasi pada `ProductService`:

```
static Future<Product> fetchProductDetail(int id, {String? customUrl, bool simulateError = false}) as
  final endpoint = simulateError ? '$errorUrl/$id' : (customUrl ?? '$baseUrl/products/$id');
  final response = await client.get(Uri.parse(endpoint)).timeout(const Duration(seconds: 15));

  if (response.statusCode == 200) {
    final Map<String, dynamic> jsonMap = jsonDecode(response.body);
    return Product.fromJson(jsonMap);
  } else {
    throw HttpException('Gagal memuat detail produk #$id (Status Code: ${response.statusCode})');
  }
}
```

Penerapan pada `ProductDetailPage`:

Halaman `ProductDetailPage` mendukung navigasi dengan hanya mengirimkan `productId` (misal `ProductDetailPage(productId: 1)`). Jika objek `product` utuh belum ada, halaman akan menggunakan `FutureBuilder<Product>` untuk mengambil detail langsung dari Fake Store API:

```
if (resolvedProduct == null && resolvedId ≠ null) {
  _detailFuture ??= ProductService.fetchProductDetail(resolvedId);
  return FutureBuilder<Product>(
    future: _detailFuture,
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(child: CircularProgressIndicator());
      }
      if (snapshot.hasError) {
        return Center(child: Text('Gagal memuat detail produk: ${snapshot.error}'));
      }
      return _buildProductDetailScaffold(snapshot.data!);
    },
  );
}
```

Selain itu, disediakan pula tombol aksi sinkronisasi (`Icons.sync_rounded`) pada AppBar agar pengguna dapat memperbarui detail produk langsung dari server kapan saja.

---

2. Penerapan Pull-to-Refresh (`RefreshIndicator`) pada `HomePage`

Widget `RefreshIndicator` membungkus `ListView.builder` pada `HomePage`:

```
RefreshIndicator(
  onRefresh: _handleRefresh,
  color: const Color(0xFF1976D2),
  child: ListView.builder(
    physics: const AlwaysScrollableScrollPhysics(),
    itemCount: filtered.length,
    itemBuilder: (context, index) {
      return ProductCard(product: filtered[index]);
    },
  ),
)
```

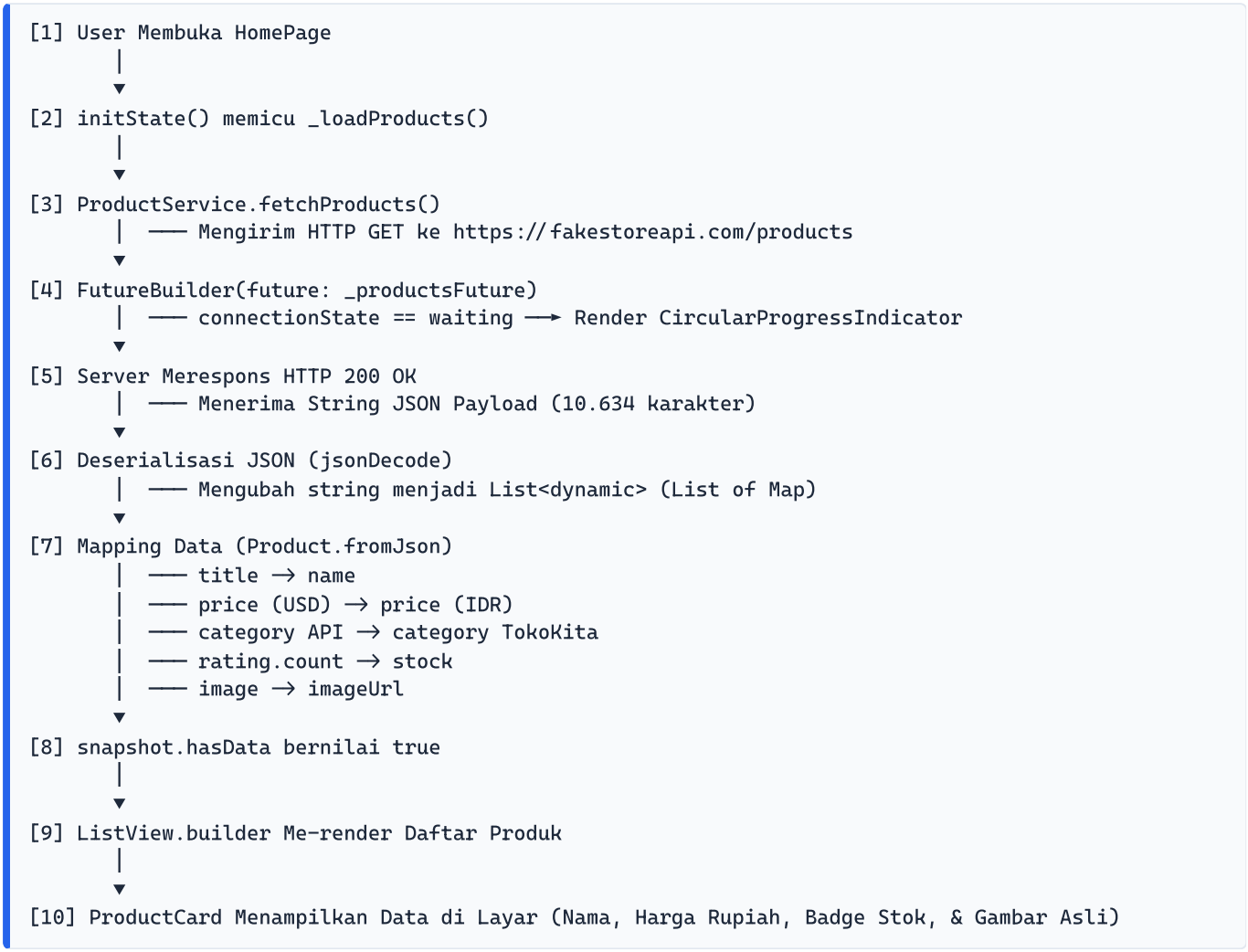
Mekanisme Kerja:

Saat pengguna menarik layar ke bawah (*pull down gesture*), `RefreshIndicator` memicu callback `_handleRefresh()`. Method ini memanggil ulang `ProductService.fetchProducts()` dan menanti respons selesai (`await _productsFuture`). Begitu data terbaru tiba, state diperbarui dan indikator refresh memudar secara mulus. `AlwaysScrollableScrollPhysics` diterapkan agar aksi pull-to-refresh tetap aktif meskipun jumlah produk sedikit atau layar belum penuh.

---

3. Alur Lengkap Request-Parsing-Tampilkan Data pada TokoKita

Berikut adalah diagram alur sekuensial dari pemanggilan pertama hingga data tampil di layar:


[1] User Membuka HomePage
[2] initState() memicu _loadProducts()
[3] ProductService.fetchProducts()
Mengirim HTTP GET ke https://fakestoreapi.com/products
[4] FutureBuilder(future: _productsFuture)
connectionState == waiting ⟶ Render CircularProgressIndicator
[5] Server Merespons HTTP 200 OK
Menerima String JSON Payload (10.634 karakter)
[6] Deserialisasi JSON (jsonDecode)
Mengubah string menjadi List<dynamic> (List of Map)
[7] Mapping Data (Product.fromJson)
title → name
price (USD) → price (IDR)
category API → category TokoKita
rating.count → stock
image → imageUrl
[8] snapshot.hasData bernilai true
[9] ListView.builder Me-render Daftar Produk
[10] ProductCard Menampilkan Data di Layar (Nama, Harga Rupiah, Badge Stok, & Gambar Asli)

4. Perbandingan Struktur JSON Fake Store API vs Class Product Pertemuan 2

Aspek / Properti	Struktur JSON Fake Store API	Struktur Class `Product` (Pertemuan 2)	Penyesuaian pada Method `Product.fromJson`
Identitas Nama	Field berupa `title` (tipe `String`)	Properti bernama `name` (tipe `String`)	Melakukan pemetaan: `name: json['title'] ?? json['name'] ?? ''`.
Harga Produk	Field `price` berupa pecahan dollar (USD, misal `109.95`)	Properti `price` berupa mata uang Rupiah (IDR, ratusan ribu/jutaan)	Membaca tipe data secara aman dengan `(json['price'] as num).toDouble()`. Untuk nilai dollar (< 1000), dikalikan estimasi kurs Rp 15.000 agar serasi dengan `formatRupiah()`.
Katalog Gambar	Field berupa `image` (URL string CDN)	Properti berupa `imageUrl` (nullable `String?`)	Melakukan pemetaan: `imageUrl: json['image'] ?? json['imageUrl']`.
Ketersediaan Stok	**Tidak ada field stok** pada Fake Store API	Properti `stock` berstatus non-nullable `int`	Menghindari error null safety dengan mengambil nilai `rating['count']` atau nilai default `20` sebagai fallback.
Kategori	String bahasa Inggris huruf kecil (*men's clothing, electronics*)	Kategori lokal TokoKita (*Elektronik, Fashion, Makanan*)	Dibuat fungsi helper `_normalizeCategory()` untuk memetakan kategori API ke chip filter TokoKita.
Deskripsi	Field `description` (`String`)	Properti `description` (nullable `String?`)	Dipetakan langsung: `description: json['description']`.

---

5. Analisis Urgensi Pengecekan `response.statusCode` vs Blok `catch`

Dalam arsitektur networking HTTP di Dart, **blok `catch` HANYA akan dieksekusi apabila terjadi exception teknis pada level transport/koneksi**, seperti:
- Tidak ada koneksi internet sama sekali (`SocketException`).
- Request mengalami timeout karena latency tinggi (`TimeoutException`).
- URL salah format atau server DNS gagal ditemukan (`FormatException`).

Sebaliknya, apabila koneksi fisik ke server berhasil tersambung, tetapi endpoint tidak ditemukan (**HTTP 404 Not Found**) atau server mengalami error internal (**HTTP 500 Internal Server Error**), library `http` **TIDAK AKAN melempar exception**. Server tetap mengirimkan paket HTTP yang sah secara protokol, dengan header status code `404` atau `500` dan body yang biasanya berupa teks HTML error atau pesan JSON singkat seperti:

```
{"status": "error", "message": "Product not found"}
```

Jika aplikasi TokoKita **tidak memeriksa** `response.statusCode == 200` dan langsung mengeksekusi `jsonDecode(response.body)`:
1. Aplikasi akan mencoba mem-parsing pesan error 404/500 sebagai daftar produk (`List<Product>`).
2. Proses ini akan memicu *unhandled runtime exception* (seperti `type 'String' is not a subtype of type 'List<dynamic>'` atau `NoSuchMethodError`).
3. Akibatnya, antarmuka aplikasi crash atau menampilkan layar merah (*red screen of death*).

Oleh karena itu, pemeriksaan `if (response.statusCode == 200)` adalah **lapisan validasi gerbang utama (*guard clause*)** untuk memastikan bahwa data yang diterima benar-benar payload produk yang valid sebelum dialirkan ke parser JSON. Jika status code bukan 200, aplikasi dapat secara sengaja melempar `HttpException` yang ditangkap rapi oleh `FutureBuilder` untuk menampilkan antarmuka error yang elegan kepada pengguna.

G. Hasil Pengujian Otomatis (`flutter test` & `flutter analyze`)

Seluruh rangkaian pengujian komprehensif dijalankan pada lingkungan proyek `tokokita`:

```
00:03 +17: All tests passed!
```

1. `flutter analyze`: **No issues found!** (0 error, 0 warning, 0 lint issue).
2. `flutter test`: **17 dari 17 unit & widget tests lulus 100%**:
3. 8 test mencakup pengujian Pertemuan 7 (parsing JSON Fake Store API, serialisasi `toJson`, real HTTP request logic, mock client, penanganan error status code, `FutureBuilder` state, tombol "Coba Lagi", `fetchProductDetail`, dan `RefreshIndicator`).
4. 9 test mencakup modul Pertemuan 1 s.d. 6 (`CartController`, `RxList`, `Obx`, navigasi `Get.to` / `Get.back`, `CartPage`, `CheckoutPage`, model `Product`, `DiscountedProduct`, dan `formatRupiah`).

H. Kesimpulan

Praktikum Pertemuan 7 berhasil mentransformasi aplikasi **TokoKita** dari aplikasi berbasis data statis menjadi aplikasi toko daring dinamis yang terhubung langsung dengan layanan REST API.

Pencapaian utama praktikum ini meliputi:
1. **Pemisahan Tanggung Jawab (*Separation of Concerns*)**: Logika networking diisolasi di dalam `ProductService`, sehingga kode UI pada `HomePage` dan `ProductDetailPage` tetap bersih dan modular.
2. **Robustness & Error Resilience**: Penerapan `try-catch` terpadu bersama validasi `response.statusCode` berhasil melindungi aplikasi dari potensi crash akibat gangguan sinyal maupun error server.
3. **Pengalaman Pengguna (*User Experience*) yang Modern**: Penggunaan `FutureBuilder`, `CircularProgressIndicator`, `RefreshIndicator`, dan tombol retry menyajikan umpan balik visual yang responsif.
4. **Sinkronisasi Sempurna dengan Pertemuan 1-6**: Objek `Product` hasil konsumsi REST API langsung kompatibel 100% dengan widget `ProductCard` (Pertemuan 4), sistem keranjang reaktif GetX `CartController` (Pertemuan 6), dan alur checkout pesanan.